

Creational Design Patterns

Creational design patterns deal with **object creation mechanisms**, trying to create objects in a manner suitable to the situation. Their main goal is to **decouple the client from the process of creating objects**.

List of Creational Patterns

1. **Singleton** → Ensure only one instance of a class exists.
 2. **Factory Method** → Define an interface for creating objects, but let subclasses decide which class to instantiate.
 3. **Abstract Factory** → Provide an interface to create families of related or dependent objects without specifying concrete classes.
 4. **Builder** → Separate the construction of a complex object from its representation, allowing different representations.
 5. **Prototype** → Create new objects by copying existing ones (cloning).
-

Detailed Explanation

1. Singleton

- **Intent:** Only one instance of a class, global point of access.
 - **Use When:**
 - Shared resources (loggers, config, DB connection pools).
 - System-wide control points.
 - **Pros:** Consistency, saves memory.
 - **Cons:** Harder testing, can become bottleneck.
-

2. Factory Method

- **Intent:** Define an interface for creating an object, but let subclasses decide which class to instantiate.
- **Use When:**
 - You don't know beforehand which class you may need.
 - You want to delegate instantiation logic to subclasses.
- **Pros:** Loose coupling, promotes code extensibility.
- **Cons:** More classes, complexity grows.

Example (Java):

```
abstract class ShapeFactory {  
    abstract Shape createShape();  
}
```

```
class CircleFactory extends ShapeFactory {  
    Shape createShape() { return new Circle(); }  
}
```

3. Abstract Factory

- **Intent:** Provide an interface to create families of related objects without specifying their concrete classes.
- **Use When:**
 - You need to create groups/families of related products.
 - System should be independent of how objects are created.
- **Pros:** Ensures product families stay consistent.
- **Cons:** Hard to add new product types.

Example (Java):

```
interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}  
  
class WinFactory implements GUIFactory {  
    public Button createButton() { return new WinButton(); }  
    public Checkbox createCheckbox() { return new WinCheckbox(); }  
}
```

4. Builder

- **Intent:** Construct a complex object step by step, separating construction and representation.
- **Use When:**
 - Object has many optional/complex configurations.
 - Avoid telescoping constructors.
- **Pros:** Flexible, readable, step-by-step object creation.
- **Cons:** More code and classes.

Example (Java):

```
User user = new User.Builder("John")  
    .age(25)  
    .email("john@example.com")  
    .build();
```

5. Prototype

- **Intent:** Create new objects by copying existing ones.
- **Use When:**
 - Object creation is expensive (CPU/memory).
 - Need to avoid rebuilding complex objects.
- **Pros:** Faster cloning than re-creating.
- **Cons:** Complex with deep copy requirements.

Example (Java):

```
class Prototype implements Cloneable {  
    public Prototype clone() throws CloneNotSupportedException {  
        return (Prototype) super.clone();  
    }  
}
```

When to Use Which

- **Singleton** → When only one instance is required (e.g., Logger, Config Manager).
- **Factory Method** → When you want subclasses to decide which object to create.
- **Abstract Factory** → When working with families of related products (e.g., GUI toolkits for Windows/Mac).
- **Builder** → When constructing complex objects with many optional fields (e.g., Documents, Objects with many parameters).
- **Prototype** → When creating objects is costly, and cloning existing ones is faster.

Comparison of Creational Patterns

Pattern	Main Purpose	When to Use	Example Use Case
Singleton	One instance globally	Shared resources, configs	Logger, DB connection pool
Factory Method	Delegate creation to subclasses	When exact type is unknown at compile time	Shape factory, Document factory
Abstract Factory	Families of related products	When products must match consistently	GUI toolkit (Windows/Mac buttons & checkboxes)
Builder	Step-by-step construction	When object has many optional params	<code>StringBuilder</code> , Complex DTOs
Prototype	Clone existing object	When object creation is expensive	Game characters, Document templates

Summary

Creational design patterns **abstract the instantiation process**, making the system more flexible and reusable. - Use **Singleton** for global unique objects. - Use **Factory Method** for delegating creation. - Use **Abstract Factory** for related product families. - Use **Builder** for complex configurable objects. - Use **Prototype** for cloning costly objects.
